PL/SQL Training

SESSION 2: WRITING EXECUTABLE STATEMENTS & CONTROL STRUCTURES

DAVA.X ACADEMY SESSIONS

Svetlana Sura & Iulian Aurel Cozma

10–20 JUNE 2025

endava

SESSION 1

Writing Executable Statements

- SQL Functions in PL/SQL
- Nested Blocks
- Variable Scope and Visibility
- Write readable code with appropriate indentations

Writing executable statements

PL/SQL block syntax and guidelines:

Because PL/SQL is an extension of SQL, general SQL syntax rules also apply to PL/SQL.

A line of PL/SQL code contains groups of characters known as lexical units:

- Delimiters (simple and compound symbols)
- Identifiers (here we include also reserved words)
- Literals
- Comments

To improve readability lexical units are separated by spaces.

You can't embed spaces in lexical units except for literals and comments.

Statement can be split across lines, but keywords must not be split.

Delimiters

Simple Symbols

Symbol	Meaning
+	Addition operator
-	Subtraction/negation operator
*	Multiplication operator
/	Division operator
=	Relational operator
@	Remote access indicator
;	Statement terminator

Compound Symbols

Symbol	Meaning
<>	Relational operator
!=	Relational operator
||	Concatenation operator
--	Single line comment indicator
/*	Beginning comment delimiter
*/	Ending comment delimiter
:=	Assignment operator

Literals

- **Character and date literals must be enclosed in single quotation marks.**

```
v_name := 'Henderson';
```

- **Numbers can be simple values or scientific notation.**

Commenting Code

- **Prefix single-line comments with two dashes (--).**
- **Place multiple-line comments between the symbols /* and */.**

Writing executable statements

endava

SQL Functions in PL/SQL

- Available in procedural statements:
  - Single-row number
  - Single-row character
  - Data type conversion
  - Date
  - Timestamp
  - GREATEST and LEAST
  - Miscellaneous functions

} Same as in SQL

- Not available in procedural statements:
  - DECODE
  - Group functions

Operators in PL/SQL

- Logical
- Arithmetic
- Concatenation
- Parentheses to control order of operations

} Same as in SQL

- Exponential operator (**)

Examples:

- Increment the counter for a loop.

```
v_count      := v_count + 1;
```

- Set the value of a Boolean flag.

```
v_equal      := (v_n1 = v_n2);
```

- Validate if an employee number contains a value.

```
v_valid      := (v_empno IS NOT NULL);
```

SQL Functions in PL/SQL: Examples

- Get the length of a string:

```
desc_size INTEGER(5);
prod_description VARCHAR2(70):='You can use this product ';

-- get the length of the string in prod_description
desc_size:= LENGTH(prod_description);
```

- Convert the employee name to lowercase:

```
emp_name:= LOWER(emp_name);
```

Data Type Conversion

Convert data to comparable data types

- Are of two types:
  - Implicit conversions
  - Explicit conversions

Some conversion functions:

- `TO_CHAR`
- `TO_DATE`
- `TO_NUMBER`
- `TO_TIMESTAMP`

Data Type Conversion

1
```
date_of_joining DATE:= '02-Jun-2025';
```

2
```
date_of_joining DATE:= 'June 02,2025';
```

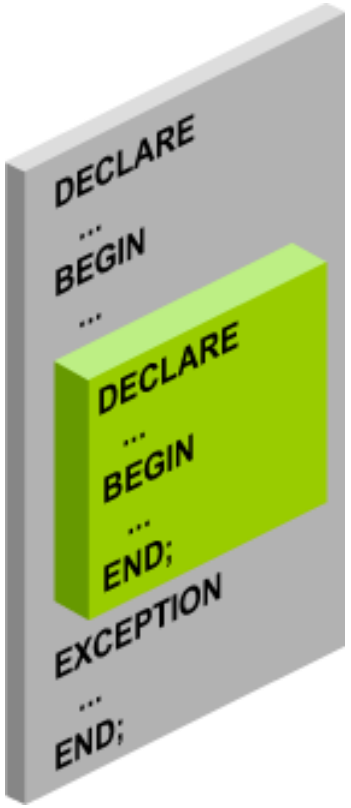
3
```
date_of_joining DATE:= TO_DATE('June 02,2025','Month DD, YYYY');
```

Nested Blocks

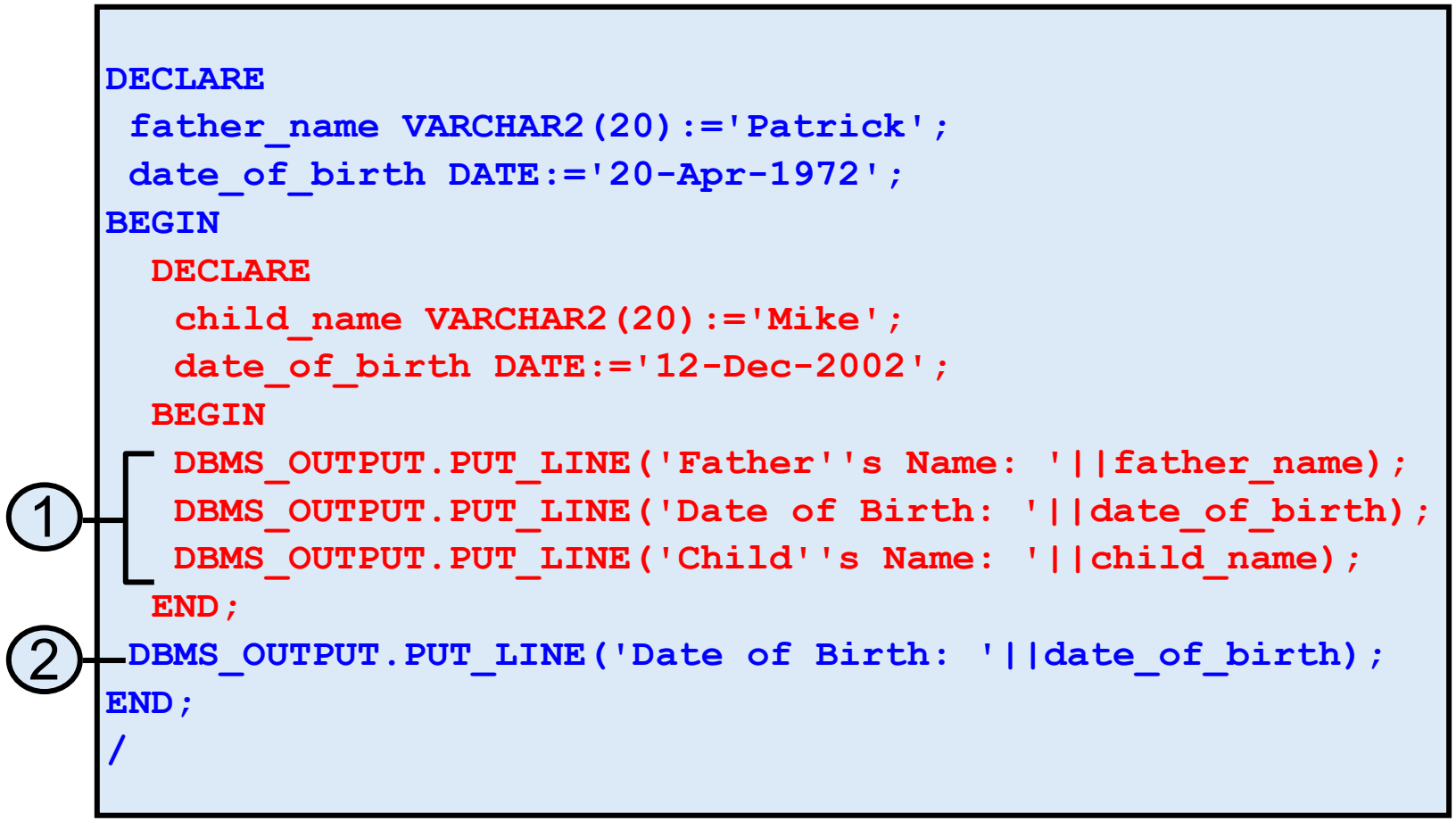
PL/SQL blocks can be nested.

- An executable section (`BEGIN ... END`) can contain nested blocks.
- An exception section can contain nested blocks.

```
DECLARE
 outer_variable VARCHAR2(20):='GLOBAL VARIABLE';
BEGIN
  DECLARE
   inner_variable VARCHAR2(20):='LOCAL VARIABLE';
  BEGIN
   DBMS_OUTPUT.PUT_LINE(inner_variable);
   DBMS_OUTPUT.PUT_LINE(outer_variable);
  END;
 DBMS_OUTPUT.PUT_LINE(outer_variable);
END;
/
```



Variable Scope and Visibility



Indenting Code

- For clarity and readability, indent each level of code
- To show structure, you can divide line by using carriage return and indent lines using spaces or tabs

```
BEGIN
    -- good
    IF x = 1 THEN
        y:= 1;
    ELSE
        y:=0;
    END IF;

    -- bad
    IF x = 1 THEN y:= 1; ELSE y:=0; END IF;
END;
/
```

```
DECLARE
    myvar   NUMBER;
BEGIN
    -- good
    SELECT c3
      INTO myvar
      FROM TAB1 t1,
           TAB2 t2
     WHERE t1.c1 = t2.c1
       AND t2.c2 = 5;

    -- great
    SELECT c3
      INTO myvar
      FROM TAB1 t1
      JOIN TAB2 t2 ON (t1.c1 = t2.c1)
     WHERE t2.c2 = 5;

    -- bad
    SELECT c3 INTO myvar FROM TAB1 t1, TAB2 t2 WHERE t1.c1 = t2.c1 AND t2.c2 = 5;
END;
/
```

Indenting Code

For clarity, indent each level of code.

Example:

```
BEGIN
   IF x=0 THEN
      y:=1;
   END IF;
END;
/
```

```
DECLARE
   deptno        NUMBER(4);
   location_id   NUMBER(4);
BEGIN
   SELECT   department_id,
            location_id
   INTO     deptno,
            location_id
   FROM     departments
   WHERE    department_name
            = 'Sales';
...
END;
/
```

SESSION 2 :

Writing Control Structures

- Identify the uses and types of control structures
- Construct an `IF` statement
- Use `CASE` statements and `CASE` expressions
- Construct and identify different loop statements
- Make use of guidelines while using the conditional control structures

IF Statements

endava

The IF statement enables you to implement conditional branching logic in your programs.

IF Type	Characteristics
IF THEN END IF;	This is the simplest form of the IF statement. The condition between IF and THEN determines whether the set of statements between THEN and END IF should be executed. If the condition evaluates to FALSE or NULL, the code will not be executed.
IF THEN ELSE END IF;	This combination implements either/or logic: based on the condition between the IF and THEN keywords, execute the code either between THEN and ELSE or between ELSE and END IF. One of these two sections of statements is executed.
F THEN ELSIF ELSE END IF;	This last and most complex form of the IF statement selects a condition that is TRUE from a series of mutually exclusive conditions and then executes the set of statements associated with that condition. If you're writing IF statements like this in any Oracle Database release from Oracle9/ Database Release 1 onward, you should consider using searched CASE statements instead.

IF Statements

Syntax:

```
IF condition THEN
  statements;
[ELSIF condition THEN
  statements;]
[ELSE
  statements;]
END IF;
```

```
DECLARE
  myage number:=&myage;
BEGIN
  IF myage < 18
  THEN
    DBMS_OUTPUT.PUT_LINE(' You are a child ');
  ELSE
    DBMS_OUTPUT.PUT_LINE(' You are an adult ');
  END IF;
END;
/
```

IF ELSIF ELSE Clause

```
DECLARE
myage number:=& myage;
BEGIN
IF myage < 11
 THEN
       DBMS_OUTPUT.PUT_LINE(' You are a child ');
   ELSIF myage < 20
     THEN
       DBMS_OUTPUT.PUT_LINE(' You are young ');
   ELSIF myage < 30
     THEN
       DBMS_OUTPUT.PUT_LINE(' You are in your twenties');
   ELSIF myage < 40
     THEN
       DBMS_OUTPUT.PUT_LINE(' You are in your thirties');
 ELSE
    DBMS_OUTPUT.PUT_LINE(' You are always young ');
END IF;
END;
/
```

CASE Expressions

The CASE statement and expression offer another way to implement conditional branching. By using CASE instead of IF, you can often express conditions more clearly and even simplify your code. There are two forms of the CASE statement:

Simple CASE statement. This one associates each of one or more sequences of PL/SQL statements with a value. The simple CASE statement chooses which sequence of statements to execute, based on an expression that returns one of those values.

Searched CASE statement. This one chooses which of one or more sequences of PL/SQL statements to execute by evaluating a list of Boolean conditions. The sequence of statements associated with the first condition that evaluates to TRUE is executed.

Simple CASE statements	Searched CASE statements
CASE expression WHEN result1 THEN statements1 WHEN result2 THEN statements2 ... ELSE statements_else END CASE;	CASE WHEN expression1 THEN statements1 WHEN expression2 THEN statements2 ... ELSE statements_else END CASE;

Iterative processing with loops

Loops in code give you a way to execute the same body of code more than once.

There are three loop types:

Basic Loops	`WHILE` Loops	`FOR` Loops
Syntax	Syntax	Syntax

`LOOP`
`  statement1;`
`  . . .`
`  EXIT [WHEN condition];`
`END LOOP;`
	`WHILE condition`
	`LOOP`
	`  statement1;`
	`  statement2;`
	`  . . .`
	`END LOOP;`
		`FOR counter IN [REVERSE]`
		`  lower_bound..upper_bound LOOP`
		`  statement1;`
		`  statement2;`
		`  . . .`
		`END LOOP;`

It's called *simple LOOP* for a reason: It starts simply with the LOOP keyword and ends with the END LOOP statement. The loop will terminate if you execute an EXIT, EXIT WHEN, or RETURN within the body of the loop (or if an exception is raised).	Use the `WHILE` loop to repeat statements while a condition is `TRUE`.	Use a `FOR` loop to shortcut the test for the number of iterations. Do not declare the counter; it is declared implicitly.

Guidelines While Using Loops

Use the basic loop when the statements inside the loop must execute at least once.

Use the `WHILE` loop if the condition has to be evaluated at the start of each iteration.

Use a `FOR` loop if the number of iterations is known.

Nested Loops and Labels

Nest loops to multiple levels.

Use labels to distinguish between blocks and loops.

Exit the outer loop with the EXIT statement that references the label.

```
...
BEGIN
  <<Outer_loop>>
  LOOP
    counter := counter+1;
  EXIT WHEN counter>10;
    <<Inner_loop>>
    LOOP
      ...
      EXIT Outer_loop WHEN total_done = 'YES';
      -- Leave both loops
      EXIT WHEN inner_done = 'YES';
      -- Leave inner loop only
      ...
    END LOOP Inner_loop;
    ...
  END LOOP Outer_loop;
END;
/
```

Control structures

GOTO and NULL Statements:

GOTO - branches to a label unconditionally.

- label must be unique and precede an executable statement or a PL/SQL block.

- can't branch into **IF** statement, **CASE** statement, **LOOP** statement or sub-block.

NULL - does nothing, just passes control to the next statement.

- used to improve readability by making the meaning and action of condition statements clear.

- can be used in exception handling indicating that you are aware of a possibility to get that error, but no action is necessary.

```
BEGIN

    ....

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        NULL;
END;
```

```
BEGIN

    statement1;

    if condition then
        goto label1;
    end if;

    statement2;

    <<label1>>
    statement3;

END;
```

Summary & Homework

THEORY

- PL/SQL 101 Part 1-5
- **CASE in PL/SQL**

PRACTICE

HOMEWORK

- **Quizz Beginners**
- **Quiz on Case Expressions**
- **Quiz on Loops in PL/SQL**
- **Quiz on The WHILE-LOOP Statement**